

Aplicação K-Means

Projeto Concorrente I

Rary Emanuel Gonçalves Coringa

DIM0124 - Programação Concorrente

Departamento de Informática e Matemática Aplicada

Universidade Federal do Rio Grande do Norte

Introdução

Implementação Serial

Concorrente v2 — Lotes e Workers

Concorrente v3 — Dados em Memória

Implementação Erlang

Comparativo Parcial

Concorrente v4 — Concorrência Estruturada

Concorrente v5 — Fork/Join

v6 — Spark (Processamento Distribuído)

Por que Não há v4, v5 ou v6 em Erlang

Comparativo Final — Visão Completa

Introdução

O Algoritmo K-Means

- Agrupa N pontos em k clusters, associando cada ponto ao centróide mais próximo
- Opera em duas etapas que se repetem:
 - **Atribuição** — cada ponto é associado ao centróide mais próximo
 - **Atualização** — cada centróide é recalculado como média dos seus pontos
- Para quando os centróides estabilizam ou o limite de iterações é atingido
- A etapa de atribuição é independente entre pontos — candidato natural à paralelização

- Pedidos sintéticos gerados artificialmente
- $\approx 5,75$ milhões de linhas $\times$ 38 features numéricas
- Tamanho em disco: ≈ 1 GB
- Tamanho carregado em memória: $\approx 1,75$ GB

Parâmetros fixos nos experimentos

$k = 10$ clusters · $\text{maxIter} = 3-20$ · 38 dimensões

Hardware

- CPU: Intel Core i5-13450HX
10 núcleos / 16 threads,
4,6 GHz
- RAM: 16 GB DDR5
dual-channel
- Disco: SSD NVMe 512 GB

Software

- Ubuntu 24.04.4 LTS
- Java: OpenJDK 25.0.1
(Temurin)
- Erlang: OTP 25, BEAM
13.2
JIT AsmJit habilitado

- Linguagem funcional, roda na máquina virtual BEAM
- Processos leves e isolados — sem memória compartilhada
- Comunicação exclusiva por troca de mensagens
- GC por processo, sem pausas globais (*stop-the-world*)

- Sem volatile, mutexes ou semáforos
- Race conditions **impossíveis por construção** — cada processo tem sua própria memória
- Escala naturalmente para sistemas distribuídos sem mudar o modelo de programação

Por que usar Erlang neste projeto?

Comparar dois modelos fundamentalmente diferentes de concorrência: memória compartilhada (Java) vs. troca de mensagens (Erlang).

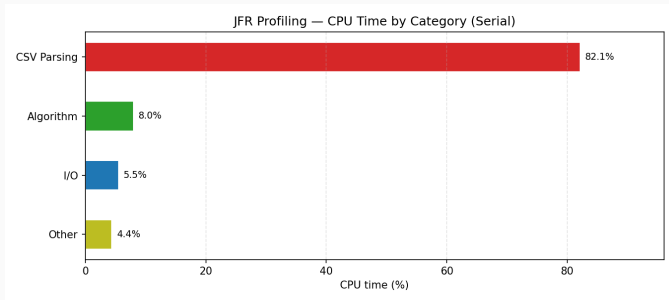
Implementação Serial

- A cada iteração, o algoritmo **relê o CSV do disco** inteiro
- Para cada linha: converte texto para números, calcula o cluster mais próximo, acumula
- Tudo em uma única thread — leitura e computação misturados

Expectativa

O gargalo deveria ser a aritmética do K-Means — é ela que é executada milhões de vezes.

Onde o Tempo é Gasto? (JFR)



82% do CPU gasto convertendo texto para número — o algoritmo em si representa apenas 8%.

Microbenchmark (JMH)

- Execução completa: 212 s
- Custo por iteração: ≈ 12 s
- `closestCentroid`: 96 ns/chamada
- `squaredDistance`: 9 ns/chamada

Diagnóstico

O gargalo não é a CPU — é a conversão do CSV a cada iteração. Paralelizar a aritmética sozinha não vai ajudar.

Macrobenchmark (JMeter)

- 1 thread: 1,56 exec/min
- 16 threads: 4,66 exec/min
- Ganho de $\approx 3\times$ com $16\times$ mais threads
- GC: $<1\%$ do tempo total

Comparação de Coletores de GC — Throughput

Três coletores testados com 1, 2, 4, 8 e 16 threads JMeter (implementação serial):

Coletor	1 thread	16 threads
G1GC (padrão JDK 25)	1,56 exec/min	4,66 exec/min
ZGC	1,50 exec/min	4,40 exec/min
ParallelGC	1,52 exec/min	4,85 exec/min

Comparação de Coletores de GC — Conclusão

- **ParallelGC** lidera levemente em alta carga — otimizado para vazão
- **ZGC** fica um pouco atrás — suas barreiras de escrita têm custo extra
- **G1GC** equilibrado, entre os dois
- Pausas de GC somam **<1%** do tempo em todos os casos

O padrão se repete nas variantes v2

Coletor de GC não é o gargalo. A escolha tem impacto marginal no throughput.

Concorrente v2 — Lotes e Workers

O que Mudou na v2

- Pontos agrupados em lotes de 5.000 e distribuídos para workers em paralelo
- Cada worker acumula em arrays **locais** — sem sincronização necessária
- Thread coordenadora reduz os resultados após todos os workers terminarem

Três variantes testadas:

- **v2-platform** — workers com threads reais do sistema operacional
- **v2-virtual** — workers com virtual threads (Java 21+)
- **v2-hybrid** — workers reais + coordenador virtual

Abordagem	B2 (s/op)	Erro
Serial	60,05	$\pm 2,46$
v2-platform	60,69	$\pm 6,41$
v2-virtual	71,61	$\pm 227,05$
v2-hybrid	60,44	$\pm 5,33$

Nenhuma melhoria no tempo por iteração

A leitura sequencial do CSV continua dominando. Paralelizar apenas a aritmética (8% do tempo) não move o resultado.

Resultados v2 — Macrobenchmark por Coletor

Throughput por coletor de GC (v2-platform, mesma tendência nas outras variantes):

Coletor	1 thread	16 threads
G1GC	1,65 exec/min	4,49 exec/min
ZGC	1,41 exec/min	4,13 exec/min
ParallelGC	1,60 exec/min	4,39 exec/min

ZGC levemente atrás, ParallelGC e G1GC equivalentes — **mesmo padrão da versão serial**. Nenhum coletor resolve o gargalo real: a leitura do CSV.

- Leituras concorrentes dos centróides: ✓ seguras
- Escrita em array compartilhado sem proteção: ✗ até 28% de atualizações perdidas
- Solução adotada: cada worker acumula em arrays próprios

Insight da v2-hybrid

O coordenador virtual suspende durante leitura de arquivo, tornando o CSV invisível ao profiler — revelando que o problema era a leitura, não a coordenação.

Concorrente v3 — Dados em Memória

Problema identificado

A cada iteração, o algoritmo lê e converte ≈ 1 GB de CSV. Isso acontece 3 a 20 vezes por execução.

Solução: carregar todo o dataset em memória **uma única vez** antes das iterações.

- Dataset lido na inicialização e mantido como array de doubles
- Iterações operam diretamente sobre o array — sem disco
- Array dividido em N partes (uma por processador lógico)
- Workers processam em paralelo, coordenador aguarda todos com `CountDownLatch`

- Custo por iteração: 60 s $\rightarrow$ 1,6 s (37 $\times$ mais rápido)
- Variância colapsa: ± 227 s $\rightarrow$ $\pm 0,17$ s — iterações previsíveis
- Perfil JFR: nenhum evento de leitura de arquivo durante o clustering

Trade-off de memória

- Heap: 250 MB $\rightarrow$ 2,7 GB
- GC: 1.000 coletas $\rightarrow$ apenas 60 coletas
- Dataset precisa caber inteiro na RAM

- 1 thread JMeter: 1,6 → 52,5 exec/min (32×)
- 16 threads JMeter: 4,5 → 62,3 exec/min (14×)

Por que o ganho diminui com mais threads?

Com tudo em memória, cada execução já ocupa todos os núcleos internamente. Threads JMeter adicionais competem pelos mesmos recursos.

Comparação direta com a versão serial

Serial: 1,56 exec/min com 1 thread → v3: 52,5 exec/min com 1 thread

Implementação Erlang

Por que Não há Versão Serial

- Erlang é construída em torno do modelo de atores: processos são a unidade fundamental de execução
- Toda computação significativa acontece por troca de mensagens
- Escrever um K-Means verdadeiramente serial forçaria a ignorar os mecanismos centrais da linguagem

Decisão

A implementação Erlang parte diretamente da versão concorrente com dados em memória, o equivalente à v3 em Java.

Como Funciona — Modelo de Atores

- Dataset em memória, 16 workers BEAM — um por scheduler disponível
- Cada worker mantém seu pedaço dos dados localmente desde a inicialização

A cada iteração:

1. Processo principal envia os centróides para cada worker
2. Workers calculam o cluster mais próximo e acumulam resultados parciais
3. Workers enviam os parciais de volta ao processo principal
4. Processo principal agrega e recalcula os centróides

Resultados Erlang vs Java v3

Benchmark	Java v3	Erlang	Fator
closestCentroid	96 ns	15.976 ns	165×
squaredDistance	9 ns	1.053 ns	113×
5 iterações completas	1,6 s	106,3 s	66×

Ambas as implementações usam dados em memória e 16 workers.
A diferença é inteiramente na **velocidade da aritmética por ponto**.

Por que Erlang é Mais Lento?

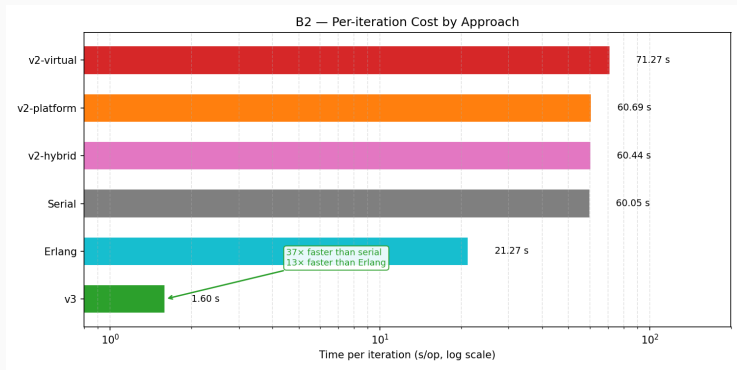
- **Aritmética:** JVM otimiza loops com vetorização SIMD; BEAM processa cada valor individualmente
- **Ociosidade:** schedulers com 45,7% de utilização — workers ficam parados enquanto o processo principal agrega os resultados
- **GC:** 2,13 milhões de coletas (Erlang) vs 60 (Java) — cada ponto processado gera objetos temporários na BEAM

Estrutural, não acidental

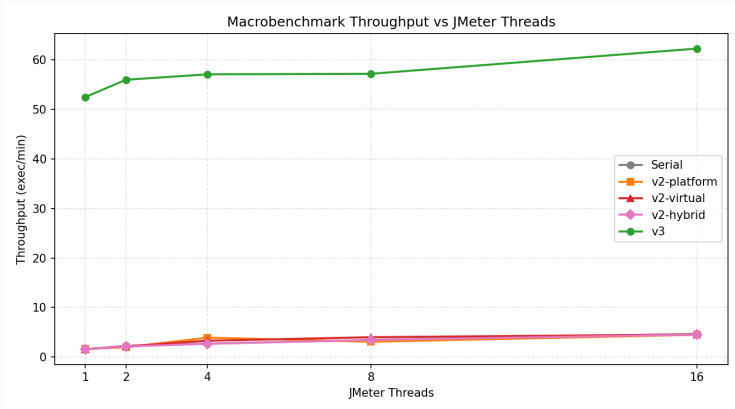
Esses custos são inerentes ao modelo de atores para cargas CPU-bound. Erlang não foi projetada para este tipo de workload.

Comparativo Parcial

Custo por Iteração — Todas as Abordagens



Evolução do Throughput



Painel superior: Serial e v2 (escala 0–7). Painel inferior: v3 (escala 0–70).

Trade-off: Releitura do CSV vs Dados em Memória

Releitura a cada iteração (v1/v2)

- ✓ Baixo consumo de memória (≈ 250 MB)
- ✓ Dataset pode ser maior que a RAM
- ✗ 82% do CPU gasto em conversão de texto
- ✗ ≈ 12 s por iteração

Dataset em memória (v3)

- ✓ Iterações $37\times$ mais rápidas
- ✓ CPU dedicado 100% ao algoritmo
- ✗ Heap de $\approx 2,7$ GB necessário
- ✗ Dataset deve caber na RAM

Lição

O JFR revelou onde o tempo realmente ia. Sem medir, paralelizaríamos os 8% do algoritmo e ignoraríamos os 82% do CSV.

Quando Cada Modelo Vence

Java (memória compartilhada)

- ✓ Aritmética 113–165× mais rápida
- ✓ Todos os núcleos ativos durante iteração
- ✓ Muito menos pressão de GC
 - Ideal para cargas CPU-bound

Erlang (modelo de atores)

- ✓ Race conditions impossíveis por construção
- ✓ GC sem pausas globais
- ✓ Escala para sistemas distribuídos
 - Ideal para I/O intensivo e alta disponibilidade

K-Means é CPU-bound com pouca comunicação entre iterações —
o caso em que memória compartilhada tem vantagem estrutural.

Concorrente v4 — Concorrência Estruturada

O que Mudou na v4

- `StructuredTaskScope` + `ScopedValue` no lugar de `ExecutorService` + `CountDownLatch`
- Corrige um bug real da v3: exceção de um worker podia ser perdida silenciosamente
- Confinamento local por tarefa continua igual — o algoritmo não muda, só a coordenação

O que o **StructuredTaskScope** resolve

`scope.join()` propaga a exceção original e cancela as subtarefas irmãs automaticamente.

	v3	v4
B1 (s/op)	4,55	7,08
B2 (s/op)	1,60	1,98

Diagnóstico

B2 piora $\approx 24\%$ — custo das APIs de *preview*, não um problema de design.

- 1 thread: 52,5 → 45,8 exec/min
- 16 threads: 62,3 → 52,0 exec/min
- Apenas ParallelGC testado (mesma decisão para v5)

Por que caiu?

O custo extra por iteração se propaga para o throughput agregado.

- Leitura concorrente de centróides: ✓ segura
- Escrita em array compartilhado sem proteção: ✗ até 30% de atualizações perdidas

Conclusão

O confinamento local segue correto — só a camada de coordenação mudou.

Concorrente v5 — Fork/Join

O que Mudou na v5

- Particionamento fixo (v3/v4) substituído por divisão recursiva com `RecursiveTask`
- *Work-stealing*: threads ociosas roubam subtarefas de outras threads
- Limiar: $\text{pontos} / (\text{núcleos} \times 4)$ — 4 subtarefas por núcleo
- `ScopedValue` da v4 mantido sem alterações

Sobre o limiar

Sem teste de sensibilidade — é uma escolha de design, não um parâmetro ajustado empiricamente.

	v4	v5
B1 (s/op)	7,08	5,72
B2 (s/op)	1,98	1,78

Achado

Variância de B1 dispara: $\pm 16,08$ s — mesmo ruído de agendamento já visto na v2-virtual.

- 1 thread: 45,8 → 46,4 exec/min
- 16 threads: 52,0 → 52,4 exec/min
- Praticamente igual à v4 em todos os níveis de carga

Por que não melhorou?

Dataset uniforme não dá chance ao *work-stealing* de mostrar vantagem real — não há desbalanceamento entre partições para explorar.

v6 — Spark (Processamento Distribuído)

Por que um Modelo Novo

- v1–v5: threads coordenadas sobre memória compartilhada, uma única JVM
- v6: Apache Spark `local[*]`, `Dataset<Row>` particionado
- JDK 21 (não 25) — exigência do Spark 4.0

Nenhuma ferramenta antiga se aplica

JMH, JMeter e JCSstress não encaixam no modelo de execução do Spark.

- `mapToPair` + `reduceByKey` por iteração — mesmo padrão de acumulação local das versões anteriores
- *Broadcast* do snapshot de centróides a cada iteração
- Duas variantes testadas: `v6-csv` e `v6-parquet` — único código diferente é o formato de leitura

Métricas usadas

Tempo de parede + `profile JFR` (sem `JMH`, `JMeter` ou `JCStress`).

	Tempo total	Aprox./iteração
v6-csv	34.624 ms	3,46 s
v6-parquet	44.264 ms	4,43 s

Mais lento que tudo

Acima do B2 de v3/v4/v5 (1,6–2,0 s/iter) — overhead distribuído não compensa neste dataset, que cabe em uma única máquina.

Achado Curioso: Parquet foi Mais Lento

- Parquet (colunar, comprimido) $\approx 28\%$ mais lento que CSV nesta carga
- Causa: decodificar dicionário/delta custa mais CPU que ler texto bruto
- Bônus: sob pressão de memória, o cache do Spark degrada devagar — sem `OutOfMemoryError` como na v3

Mesma lição da versão serial
Medir antes de assumir.

Por que Não há v4, v5 ou v6 em Erlang

Por que Não há v4, v5 ou v6 em Erlang

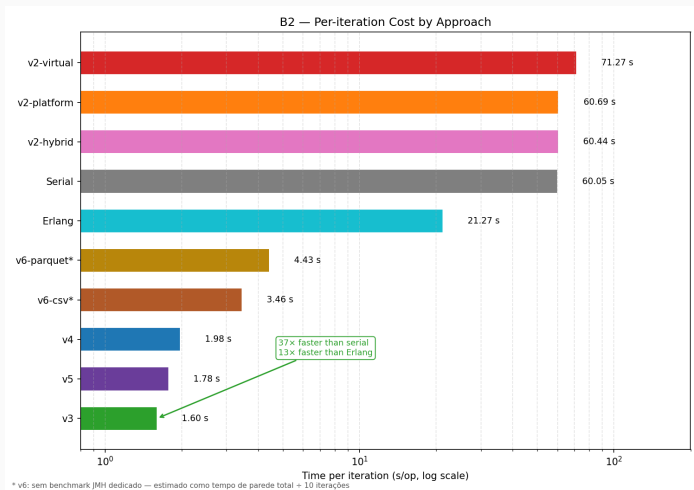
- v4 (StructuredTaskScope): coordena *threads* — Erlang não tem esse problema
- v5 (*work-stealing*): a BEAM já escalona processos sozinha
- v6 (Spark): distribui *dados*; Erlang distribui *processos* — modelos diferentes

Decisão

A implementação Erlang para na v3 (dados em memória, atores). v4–v6 resolvem problemas específicos do Java que o modelo de atores já não tem.

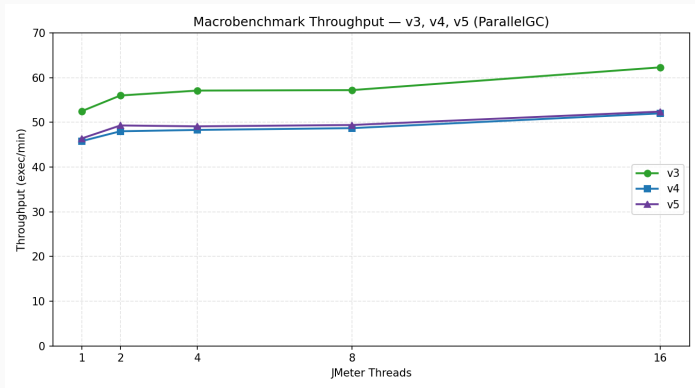
Comparativo Final — Visão Completa

Custo por Iteração — Visão Completa



v3 segue a mais rápida (1,60 s); v4/v5 ficam perto (1,98 / 1,78 s); v6 fica acima de todas ($\approx 3,46$ / 4,43 s).

Evolução do Throughput — v3, v4 e v5



Todas com ParallelGC. v6 não aparece — sem macrobenchmark JMeter para o Spark.

v4 e v5: Robustez e Work-Stealing Têm Custo

- v4 trocou um bug real (exceção perdida) por $\approx 24\%$ de custo extra por iteração
- APIs de *preview* ainda não são de graça
- v5 não superou a v4 — *work-stealing* só ajuda com carga desbalanceada, e este dataset é uniforme

Lição

Mudar o idioma de concorrência nem sempre acelera o código.

Quando Vale a Pena Distribuir

- Spark (v6) foi mais lento que todas as versões Java nesta escala — o dataset cabe em uma única máquina
- Parquet mais lento que CSV: formato de armazenamento também é uma variável de desempenho
- Cache do Spark falha diferente de uma JVM: degrada aos poucos em vez de estourar `OutOfMemoryError`

Conclusão

Processamento distribuído compensa quando os dados não cabem mais em uma única máquina — não é o caso aqui.

Obrigado!